SMS Functional Specification — Addendum 46

Purchase Order Pricing Foundation & Multi-Currency Framework

Field	Value
Document ID	SMS-FSD-ADD-046
Version	1.0
Status	Draft — Pending Approval
Author	Muhammad Ilyas
Date	2026-10-17
Parent FSD	SMS-FSD (Master)
Depends On	Addendum 45 (Supplier Pricelist & PO Auto-Pricing)
Module	SMS.Purchase, SMS.Core, SMS.Finance
Schemas	`purchase`, `core`, `finance`

---

1. Purpose

Addendum 45 introduced Supplier Pricelists and PO auto-pricing. During implementation review, seven technical gaps were identified that affect price resolution correctness, cross-module consistency, and multi-currency readiness. This addendum formally specifies each gap's solution, following Odoo's proven patterns adapted for SMS's modular monolith architecture.

Gaps Addressed:

#	Gap	Impact
1	No Order Date control	Resolver falls back to `DateTime.UtcNow` — wrong date on later edits
2	RFQ prefill ignored	"Create RFQ" button navigates but doesn't carry product/vendor/qty
3	Price precision mismatch	Pricelist at 4 decimals, PO line at 2 — silent rounding on assignment
4	Client-only re-pricing	Server doesn't re-resolve on qty change; auto-POs get Manual source
5	Import lines unpriced	Lines from Purchase Requests / Quotations skip the resolver
6	Wrong currency default	Pricelist defaults to org currency instead of vendor's currency
7	No query plan verified	Covering index designed but never benchmarked with actual data

2. Change Summary

ID	Change	Type	Priority	Gap
C1	PO Header: Add `OrderDate` field	Schema + Migration	P0	1
C2	RFQ Prefill Parameter Consumption	UI Fix	P1	2
C3	PO Line Price Precision Alignment	Schema + Migration	P0	3
C4	Server-Side Price Resolution on PO Line Mutation	Backend Service	P0	4
C5	PO Line Pricing Factory	Backend Service	P0	4,5
C6	Vendor Default Purchase Currency	Schema + Migration	P0	6
C7	Pricelist Currency Default from Vendor	UI + Backend	P1	6
C8	Exchange Rate Table	Schema + Migration	P0	6
C9	Currency Conversion Service	Backend Service	P0	6
C10	Price Resolver: Currency-Aware Resolution	Backend Service	P1	6
C11	Resolver Query Plan Verification	Performance	P1	7

3. Dependencies

Dependency	Status	Notes
Addendum 45: Supplier Pricelist	Draft	`purchase.SupplierPricelists`, `IPurchasePriceResolver`, PriceSource
`purchase.PurchaseOrders`	Existing	Receives `OrderDate` column (C1)
`purchase.PurchaseOrderLines`	Existing	`UnitPrice` precision change (C3)
`core.BusinessPartners`	Existing	Receives `DefaultPurchaseCurrencyId` (C6)
`core.Currencies`	Existing	Currency master referenced by exchange rates
Addendum 44: Financial Posting	Draft	Financial Posting Gateway — receives currency-converted amounts
RFQ Module	Existing / TBD	RFQ create screen — must consume prefill parameters (C2)
Purchase Request Module	Existing / TBD	PR-to-PO conversion — must call pricing factory (C5)

---

4. Change Details

C1 — PO Header: Add `OrderDate` Field

Problem: Addendum 45's resolver takes `OrderDate` as input, but the PO form has no `OrderDate` control. The resolver uses `DateTime.UtcNow` as a fallback, meaning a PO edited days after creation uses the wrong date for price validity checks. The C7 "order date changed" re-lookup banner (A45-0208) is designed but nothing triggers it.

Odoo Reference: Odoo's `purchase.order.date_order` is a `datetime` field defaulting to `fields.Datetime.now()`. It is editable on draft POs and frozen on confirmation. The `_select_seller` method extracts the date part: `date = order_id.date_order.date()` for pricelist validity checks.

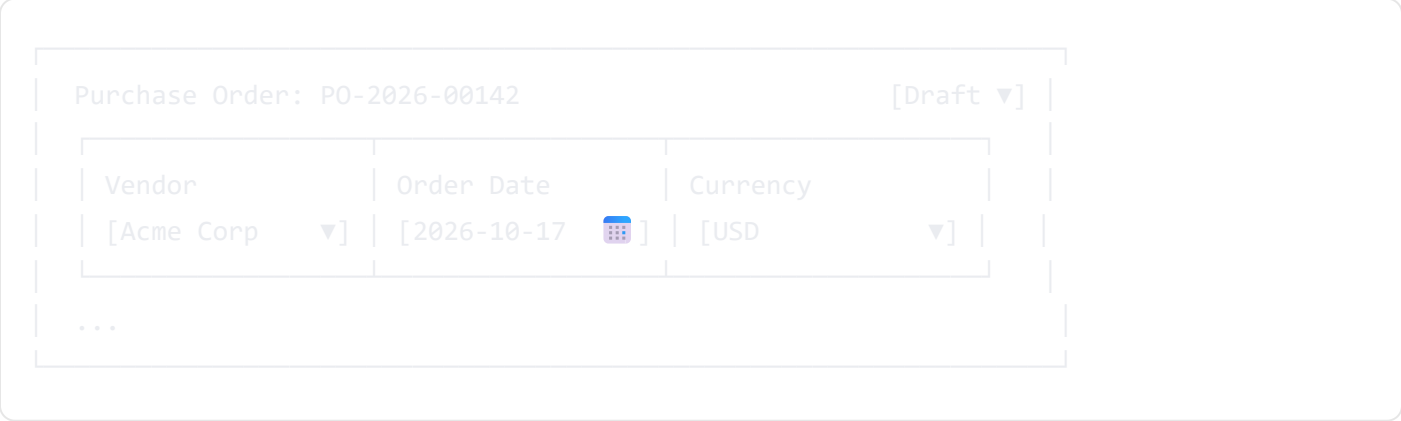
Table: `purchase.PurchaseOrders` (existing — add column)

Column	Type	Null	Default	Description
`OrderDate`	date	NO	`CAST(SYSUTCDATETIME() AS date)`	The purchase order date for pricing/terms

Behavior:

Aspect	Rule
Default	Today's date (UTC) when PO is created
Editable	Only while PO status = `Draft`
Frozen	On status transition to `Confirmed` / `Approved` — field becomes read-only
Resolver input	`PriceResolutionRequest.OrderDate` always sourced from `PO.OrderDate`, never `DateTime.UtcNow`
Re-lookup	On change → fire Addendum 45's C7 order-date banner for all lines with `PricingDateBasis = OrderDate`
Lead time calc	`ExpectedDeliveryDate = OrderDate + LeadTimeDays` (from pricelist) if buyer hasn't set one manually

UI:



C2 — RFQ Prefill Parameter Consumption

Problem: The "Create RFQ" button on the PO line warning banner (Addendum 45, C5) navigates to the RFQ creation page but that page ignores the prefill query parameters.

Odoo Reference: Odoo uses `context` dictionaries to pre-fill form values across navigation: `{'default_partner_id': vendor.id, 'default_order_line': [(0, 0, {'product_id': ..., 'product_qty': ...})]}`. SMS uses query parameters for the same purpose.

Outbound (PO Line — already designed in A45):

Navigate to: `/purchase/rfq/create?vendorId={VendorId}&productId={ProductId}&quantity={Quantity}&sourcePO={PONumber}`

Inbound (RFQ Create Page — this change):

Query Parameter	RFQ Field	Behavior
`vendorId`	Vendor dropdown	Pre-select vendor. If invalid or not found → ignore, leave empty
`productId`	First line → Product	Pre-select product on line 1. If invalid → ignore
`quantity`	First line → Quantity	Pre-fill quantity. If ⩽ 0 or non-numeric → default to 1
`sourcePO`	Reference field	Display "Referenced from PO-2026-00142" as info text (not a data link)

Validation rules:

- Each parameter is independently optional — any subset can be present
- Invalid values are silently ignored (no error toast), the field stays empty
- All pre-filled fields remain fully editable by the buyer
- The RFQ is a new document — no FK back to the source PO (that relationship is tracked later when the RFQ is completed and the price flows back)

C3 — PO Line Price Precision Alignment

Problem: `purchase.SupplierPricelists.UnitPrice` is `decimal(18,4)` but `purchase.PurchaseOrderLines.UnitPrice` is `decimal(18,2)`. When a pricelist price of $12.3456 is assigned to the PO line, it silently rounds to $12.35. The Override detection (comparing `UnitPrice` to `SystemResolvedPrice`) can false-trigger because the values never match at different precisions.

Odoo Reference: Odoo uses a unified `Product Price` decimal precision category (default 2 digits) for both `product.supplierinfo.price` and `purchase.order.line.price_unit`. Both use

`digits='Product Price'`. Odoo avoids the mismatch by using the same precision everywhere.

SAP and Dynamics 365 store prices at 4–5 decimal places internally and display at 2, which is the recommended pattern for SMS.

Solution: Align PO line `UnitPrice` to `decimal(18,4)`.

Migration M4:

```sql
-- Widen PO line UnitPrice from decimal(18,2) to decimal(18,4)
-- This is a precision increase — all existing values remain valid
-- e.g., 12.35 → 12.3500 (no data loss, no rounding)
ALTER TABLE purchase.PurchaseOrderLines
    ALTER COLUMN UnitPrice DECIMAL(18,4) NOT NULL;
```

UI Display:

- Default format mask: `#,##0.00` (2 decimal places) for standard display
- Full precision shown on hover tooltip or in detail view: `$12.3456`
- Organization-level setting (future): configurable display precision (2 or 4)

Override Detection Fix:

```csharp
// BEFORE (broken): comparing 4-decimal SystemResolvedPrice to 2-decimal UnitPrice
bool isOverride = line.UnitPrice != line.SystemResolvedPrice;

// AFTER (correct): both at 4 decimals — direct comparison works
bool isOverride = line.UnitPrice != line.SystemResolvedPrice;
// No rounding needed because both columns are now decimal(18,4)
```

C4 — Server-Side Price Resolution on PO Line Mutation

Problem: Price re-resolution on quantity or date changes runs only on the client (React). The server-side PO line create/update endpoint does not call `IPurchasePriceResolver`. This means:

- Split PO lines from partial receipts remain at `PriceSource = Manual`
- Auto-generated POs (from reorder points) get `PriceSource = Manual` with no price
- Bulk updates via API bypass price logic entirely

Odoo Reference: Odoo's `purchase.order.line._onchange_quantity()` calls `_select_seller()` server-side. Every PO line mutation (create, write, onchange) goes through the ORM which triggers the onchange chain. Even automated PO creation from `stock.rule._prepare_purchase_order_line()` calls `_select_seller()` — the pricing logic is mandatory, not optional.

Solution: Add server-side pricing middleware in the PO Line Application Service.

Interface:

```csharp
/// <summary>
/// Intercepts PO line create/update to ensure price resolution runs server-side.
/// Called by the PO Line Application Service before persisting changes.
/// </summary>
public interface IPurchaseOrderLinePricingMiddleware
{
    /// <summary>
    /// Resolves price if applicable based on what changed and the line's current PriceSou
    /// Mutates the line entity in-place.
    /// </summary>
    Task ApplyPricingAsync(PurchaseOrderLine line, PurchaseOrder po, LineChangeContext co
}

public class LineChangeContext
{
    public bool IsNew { get; set; }
    public bool ProductChanged { get; set; }
    public bool QuantityChanged { get; set; }
    public bool DeliveryDateChanged { get; set; }
    public decimal? PreviousQuantity { get; set; }
    public DateTime? PreviousDeliveryDate { get; set; }
}
```

Implementation Logic:

```
INPUT: PurchaseOrderLine line, PurchaseOrder po, LineChangeContext ctx

1. SKIP resolution if:
   - PO status ≠ Draft (no price changes on confirmed POs)
   - PriceSource = RFQ AND NOT ctx.IsNew (BR-17: RFQ prices are contractual)

2. RESOLVE price (call IPurchasePriceResolver) when:
   - ctx.IsNew AND line.UnitPrice = 0 (new line with no supplied price)
   - ctx.ProductChanged (product selection changed)
   - ctx.QuantityChanged AND PriceSource = Pricelist (silent re-resolve per BR-07)
   - ctx.DeliveryDateChanged AND current pricelist record has PricingDateBasis =
DeliveryDate

3. ON FOUND:
   line.SystemResolvedPrice = result.UnitPrice
   line.SupplierPricelistId = result.SupplierPricelistId

   IF PriceSource = Pricelist OR ctx.IsNew:
       line.UnitPrice = result.UnitPrice
       line.PriceSource = PriceSource.Pricelist
   ELSE (Override or Manual):
       // Keep buyer's price, update SystemResolvedPrice only
       // The PriceResolutionResult is returned to caller for UI toast rendering

4. ON NOT FOUND:
   line.SystemResolvedPrice = null
   line.SupplierPricelistId = null
   IF ctx.IsNew:
       line.PriceSource = PriceSource.Manual
       // line.UnitPrice stays 0 — buyer must fill before PO approval
```

Registration:

```csharp
// In PurchaseModule DI registration
services.AddScoped<IPurchaseOrderLinePricingMiddleware, PurchaseOrderLinePricingMiddlewar
```

PO Line Application Service Integration:

```csharp
```

```
public class PurchaseOrderLineAppService
{
    private readonly IPurchaseOrderLinePricingMiddleware _pricingMiddleware;

    public async Task<PurchaseOrderLine> CreateLineAsync(CreatePOLineCommand cmd)
    {
        var po = await _poRepository.GetByIdAsync(cmd.PurchaseOrderId);
        var line = MapToEntity(cmd);

        var context = new LineChangeContext
        {
            IsNew = true,
            ProductChanged = true,
            QuantityChanged = false
        };

        // Mandatory pricing step — always runs on new lines
        await _pricingMiddleware.ApplyPricingAsync(line, po, context);

        await _lineRepository.AddAsync(line);
        return line;
    }

    public async Task<PurchaseOrderLine> UpdateLineAsync(UpdatePOLineCommand cmd)
    {
        var line = await _lineRepository.GetByIdAsync(cmd.LineId);
        var po = await _poRepository.GetByIdAsync(line.PurchaseOrderId);

        var context = new LineChangeContext
        {
            IsNew = false,
            ProductChanged = cmd.ProductId != line.ProductId,
            QuantityChanged = cmd.Quantity != line.Quantity,
            DeliveryDateChanged = cmd.ExpectedDeliveryDate != line.ExpectedDeliveryDate,
            PreviousQuantity = line.Quantity,
            PreviousDeliveryDate = line.ExpectedDeliveryDate
        };

        ApplyFieldChanges(line, cmd);

        // Mandatory pricing step — runs on every update
```

```
        await _pricingMiddleware.ApplyPricingAsync(line, po, context);

        await _lineRepository.UpdateAsync(line);
        return line;
    }
}
```

C5 — PO Line Pricing Factory

Problem: Multiple code paths create PO lines — manual entry, Purchase Request conversion, Quotation/RFQ acceptance, reorder-point auto-PO generation, and CSV import. Only manual entry currently calls the price resolver. Other paths create lines with `PriceSource = Manual` and `UnitPrice = 0`.

Odoo Reference: Every PO line creation path in Odoo calls `_select_seller()`:

- `purchase.requisition._prepare_purchase_order_line()` — from purchase requisitions
- `stock.rule._prepare_purchase_order_line()` — from reordering rules / MTO
- Manual line entry — via `_onchange_product_id()`

The common pattern: `_prepare_purchase_order_line()` is the single factory method that every path uses. It always includes seller selection.

Solution: Create `IPurchaseOrderLineFactory` — the single entry point for PO line creation across all origins.

Interface:

csharp

```
/// <summary>
/// The ONLY way to create a PO line in SMS. Every code path (manual, import,
/// Purchase Request, RFQ, auto-PO) MUST use this factory. It ensures price
/// resolution runs and all tracking fields are set consistently.
/// </summary>
public interface IPurchaseOrderLineFactory
{
    /// <summary>
    /// Creates a fully-priced PO line. If suppliedPrice is provided (from RFQ,
    /// blanket order, or import), it is used directly. Otherwise, the factory
    /// calls IPurchasePriceResolver to auto-resolve the price.
    /// </summary>
    Task<PurchaseOrderLineResult> CreateLineAsync(PurchaseOrder po, CreateLineInput input
}

public class CreateLineInput
{
    public int ProductId { get; set; }
    public decimal Quantity { get; set; }
    public DateTime? ExpectedDeliveryDate { get; set; }

    // Optional: supplied by the calling code path when price is already known
    public decimal? SuppliedPrice { get; set; }
    public PriceSource? SuppliedPriceSource { get; set; }   // RFQ, Manual, etc.
    public int? SourceRfqLineId { get; set; }               // FK to RFQ line if from RFQ
    public string SuppliedOverrideReason { get; set; }       // e.g., "Imported from CSV"
}

public class PurchaseOrderLineResult
{
    public PurchaseOrderLine Line { get; set; }
    public bool PriceWasResolved { get; set; }
    public string WarningMessage { get; set; }  // null if resolved, warning if not
}
```

Behavior by Origin:

Origin	SuppliedPrice	SuppliedPriceSource	Factory Behavior
Manual entry (UI)	null	null	Call resolver. Found → Pricelist. Not found → Manual, UnitPrice = 0
Purchase Request → PO	null	null	Call resolver. Found → Pricelist. Not found → Manual, UnitPrice = 0, warn
RFQ/Quotation → PO	quoted price	RFQ	Use supplied price. PriceSource = RFQ. No resolver call
Blanket Order → Call-off	locked price	BlanketOrder (4)	Use supplied price. PriceSource = BlanketOrder. No resolver call
Reorder Point → Auto-PO	null	null	Call resolver. Found → Pricelist. Not found → Manual, UnitPrice = 0
CSV/Excel Import	imported price or null	Manual	If price > 0: use it, PriceSource = Manual. If price = 0 or null: call resolver
PO Line Split (partial GR)	parent price	parent's source	Inherit parent. If qty changed and source = Pricelist: re-resolve

PriceSource Enum Extension:

csharp

```csharp
public enum PriceSource : byte
{
    Manual       = 0,   // No system price — buyer entered manually
    Pricelist    = 1,   // Auto-resolved from SupplierPricelist
    Override     = 2,   // Was Pricelist, buyer changed it
    RFQ          = 3,   // From completed Request for Quotation
    BlanketOrder = 4    // From locked Blanket Order / Purchase Agreement
}
```

Migration M5 — Update CK constraint:

sql

```
-- Widen PriceSource to include BlanketOrder (4)
ALTER TABLE purchase.PurchaseOrderLines
    DROP CONSTRAINT CK_POLines_PriceSource;

ALTER TABLE purchase.PurchaseOrderLines
    ADD CONSTRAINT CK_POLines_PriceSource CHECK (PriceSource IN (0, 1, 2, 3, 4));
```

Summary Warning for Batch Creation:

When multiple lines are created at once (Purchase Request conversion, import), the factory returns individual results. The calling service aggregates:

```
ⓘ 8 lines created from Purchase Request PR-2026-00089.
  5 lines priced automatically from Supplier Pricelist.
  3 lines require manual pricing (highlighted in yellow).

  [View Unpriced Lines]
```

C6 — Vendor Default Purchase Currency

Problem: When creating a Supplier Pricelist record, the `CurrencyId` defaults to the organization's base currency. If the vendor normally quotes in EUR but the org's base is USD, the buyer must manually change the currency on every pricelist entry for that vendor.

Odoo Reference: Odoo stores the vendor's default purchase currency on the partner record: `res.partner.property_purchase_currency_id`. This field is a company-dependent property — each company in a multi-company setup can have a different default currency for the same vendor. The fallback chain (per PR #193005): `supplierinfo.currency_id` → `partner.property_purchase_currency_id` → `company.currency_id`.

Table: `core.BusinessPartners` (existing — add column)

Column	Type	Null	Default	Description
`DefaultPurchaseCurrencyId`	int	YES	NULL	FK → `core.Currencies`. The currency this vendor normally quotes in. NULL = org base currency

---

Migration M6:

```sql
ALTER TABLE core.BusinessPartners
    ADD DefaultPurchaseCurrencyId INT NULL;

ALTER TABLE core.BusinessPartners
    ADD CONSTRAINT FK_BusinessPartners_PurchaseCurrency
        FOREIGN KEY (DefaultPurchaseCurrencyId)
        REFERENCES core.Currencies(Id);
```

UI — Vendor Detail Form:

Add "Default Purchase Currency" dropdown in the Purchase / Financial section of the Vendor detail form. Populated from `core.Currencies` (active only). Default: empty (inherits org base currency).

C7 — Pricelist Currency Default from Vendor

Problem: Follows from C6. Even with a vendor default currency, the Supplier Pricelist "Add Price" form still defaults `CurrencyId` to the org's base currency.

Solution: Change the default `CurrencyId` on new pricelist records.

Default Resolution Chain:

```
New Pricelist Record → CurrencyId Default:
    1. Vendor.DefaultPurchaseCurrencyId   (if set)
    2. Organization.BaseCurrencyId        (fallback)
```

Frontend Change (Product Purchase Tab, Vendor Products Tab, Standalone List):

When the "[+ Add Price]" button is clicked:

1. If the vendor is already selected (e.g., on the Vendor Products Tab, the vendor is the page context): fetch `Vendor.DefaultPurchaseCurrencyId`
2. If the vendor is selected from the dropdown: on vendor selection, update the currency dropdown default
3. The buyer can always change the currency — this is a default, not a constraint

API Change — POST `/api/purchase/supplier-pricelists`:

The `currencyId` field in the request body remains required. If the client sends `currencyId = 0` or omits it:

- Server resolves default: `Vendor.DefaultPurchaseCurrencyId ?? Org.BaseCurrencyId`
- Returns the resolved `currencyId` in the response

C8 — Exchange Rate Table

Problem: SMS has no exchange rate storage. Multi-currency price conversion (C10) and future financial postings in foreign currency require a rate lookup table.

Odoo Reference: Odoo uses `res.currency.rate` with fields: `name` (date), `rate` (float, 12,6), `currency_id`, `company_id`. Rates are stored as "1 unit of base currency = X units of this currency." Lookup rule: most recent rate on or before the transaction date. Rates can be fetched automatically via providers (ECB, Open Exchange Rates) or entered manually.

Table: `finance.ExchangeRates`

Column	Type	Null	Default	Description
`Id`	int	NO	IDENTITY	PK
`OrgId`	int	NO	—	FK → `core.Organizations`
`CurrencyId`	int	NO	—	FK → `core.Currencies` — the foreign currency
`EffectiveDate`	date	NO	—	The date this rate applies from
`Rate`	decimal(18,6)	NO	—	Units of this currency per 1 unit of org base currency
`InverseRate`	decimal(18,6)	NO	—	Units of base currency per 1 unit of this currency (computed)
`Source`	tinyint	NO	0	0 = Manual, 1 = API (auto-fetched)
`ProviderName`	nvarchar(100)	YES	NULL	e.g., "ECB", "OpenExchangeRates"
`CreatedBy`	int	NO	—	FK → `core.Users`
`CreatedAt`	datetime2(7)	NO	SYSUTCDATETIME()	—

Indexes:

Name	Columns	Type	Include
`IX_ExchangeRates_Lookup`	`OrgId, CurrencyId, EffectiveDate DESC`	Non-clustered	`Rate, InverseRate`
`UQ_ExchangeRates_NoDuplicate`	`OrgId, CurrencyId, EffectiveDate`	Unique	—

Constraints:

```sql
CONSTRAINT CK_ExchangeRates_Rate       CHECK (Rate > 0),
CONSTRAINT CK_ExchangeRates_InvRate    CHECK (InverseRate > 0),
CONSTRAINT CK_ExchangeRates_Source     CHECK (Source IN (0, 1))
```

Rate Convention:

SMS follows the **"base = 1"** convention (same as Odoo):

- Org base currency: USD
- Rate for EUR on 2026-10-17: `Rate = 0.9200` means 1 USD = 0.92 EUR
- `InverseRate = 1.0870` means 1 EUR = 1.087 USD

Rate Lookup Rule:

```sql
-- Get the effective rate for a given currency on a given date
-- Returns the most recent rate on or before the requested date
SELECT TOP 1 er.Rate, er.InverseRate
FROM finance.ExchangeRates er
WHERE er.OrgId = @OrgId
  AND er.CurrencyId = @CurrencyId
  AND er.EffectiveDate <= @TransactionDate
ORDER BY er.EffectiveDate DESC;
```

If no rate exists for the currency on or before the date → return NOT FOUND. The caller must handle this (show warning, block transaction, or use rate = 1.0 for same-currency).

Migration M7 — DDL:

```sql
```

```sql
CREATE TABLE finance.ExchangeRates
(
    Id              INT             IDENTITY(1,1) NOT NULL,
    OrgId           INT             NOT NULL,
    CurrencyId      INT             NOT NULL,
    EffectiveDate   DATE            NOT NULL,
    Rate            DECIMAL(18,6)   NOT NULL,
    InverseRate     DECIMAL(18,6)   NOT NULL,
    Source          TINYINT         NOT NULL DEFAULT 0,
    ProviderName    NVARCHAR(100)   NULL,
    CreatedBy       INT             NOT NULL,
    CreatedAt       DATETIME2(7)    NOT NULL DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_ExchangeRates PRIMARY KEY CLUSTERED (Id),
    CONSTRAINT FK_ExchangeRates_Org      FOREIGN KEY (OrgId)      REFERENCES core.Organiz
    CONSTRAINT FK_ExchangeRates_Currency FOREIGN KEY (CurrencyId) REFERENCES core.Currenc
    CONSTRAINT CK_ExchangeRates_Rate     CHECK (Rate > 0),
    CONSTRAINT CK_ExchangeRates_InvRate  CHECK (InverseRate > 0),
    CONSTRAINT CK_ExchangeRates_Source   CHECK (Source IN (0, 1))
);

CREATE UNIQUE NONCLUSTERED INDEX UQ_ExchangeRates_NoDuplicate
    ON finance.ExchangeRates (OrgId, CurrencyId, EffectiveDate);

CREATE NONCLUSTERED INDEX IX_ExchangeRates_Lookup
    ON finance.ExchangeRates (OrgId, CurrencyId, EffectiveDate DESC)
    INCLUDE (Rate, InverseRate);
```

Auto-Fetch (Future — Hangfire Job):

Aspect	Detail
Schedule	Daily at 06:00 UTC (configurable per org)
Providers	ECB (free, EUR base, daily), Open Exchange Rates (API key, hourly)
Configuration	`core.OrganizationSettings` : `ExchangeRateProvider` , `ExchangeRateApiKey` , `ExchangeRateAutoFetch` (bool)
Behavior	Fetch → convert to base-currency convention → upsert into `finance.ExchangeRates` with `Source = 1`
Manual override	A manual entry (`Source = 0`) for the same OrgId + CurrencyId + EffectiveDate takes precedence over an auto-fetched one

---

C9 — Currency Conversion Service

Problem: No centralized currency conversion exists in SMS. Multiple modules (Purchase, Finance, future Sales) will need to convert amounts between currencies at specific dates.

Odoo Reference: Odoo's `res.currency._convert(from_amount, to_currency, company, date)` is the centralized conversion method. It fetches rates for both currencies on the given date and computes: `result = amount * (to_rate / from_rate)`. Since rates are stored as "1 base = X foreign," conversion between two non-base currencies requires going through the base.

Interface:

```csharp
```

```
/// <summary>
/// Centralized currency conversion service. Uses finance.ExchangeRates
/// for rate lookup. All modules use this for any currency conversion.
/// </summary>
public interface ICurrencyConverter
{
    /// <summary>
    /// Converts an amount from one currency to another using the rate
    /// effective on the given date.
    /// </summary>
    Task<CurrencyConversionResult> ConvertAsync(CurrencyConversionRequest request);

    /// <summary>
    /// Gets the exchange rate between two currencies on a given date.
    /// Returns null if no rate is available.
    /// </summary>
    Task<ExchangeRateResult> GetRateAsync(int orgId, int fromCurrencyId, int toCurrencyId
}

public class CurrencyConversionRequest
{
    public int OrgId { get; set; }
    public decimal Amount { get; set; }
    public int FromCurrencyId { get; set; }
    public int ToCurrencyId { get; set; }
    public DateTime EffectiveDate { get; set; }
    public int DecimalPlaces { get; set; } = 4;  // rounding precision
}

public class CurrencyConversionResult
{
    public bool Success { get; set; }
    public decimal ConvertedAmount { get; set; }
    public decimal RateUsed { get; set; }
    public DateTime RateDate { get; set; }      // the actual date the rate was from
    public string ErrorMessage { get; set; }     // null if success, "No exchange rate...
}

public class ExchangeRateResult
{
    public bool Found { get; set; }
```

```
    public decimal Rate { get; set; }            // fromCurrency → toCurrency
    public DateTime RateDate { get; set; }       // the actual effective date
}
```

Conversion Logic:

```
INPUT: Amount, FromCurrencyId, ToCurrencyId, EffectiveDate, OrgId

1. IF FromCurrencyId = ToCurrencyId:
      RETURN Amount (no conversion needed)

2. Get org base currency: baseCurrencyId = Org.BaseCurrencyId

3. IF FromCurrencyId = baseCurrencyId:
      // Direct: base → foreign
      rate = LookupRate(OrgId, ToCurrencyId, EffectiveDate)
      IF NOT FOUND → return error
      result = Amount * rate.Rate

4. ELSE IF ToCurrencyId = baseCurrencyId:
      // Direct: foreign → base
      rate = LookupRate(OrgId, FromCurrencyId, EffectiveDate)
      IF NOT FOUND → return error
      result = Amount * rate.InverseRate

5. ELSE:
      // Cross-rate: foreign → base → foreign
      fromRate = LookupRate(OrgId, FromCurrencyId, EffectiveDate)
      toRate   = LookupRate(OrgId, ToCurrencyId, EffectiveDate)
      IF EITHER NOT FOUND → return error
      baseAmount = Amount * fromRate.InverseRate
      result = baseAmount * toRate.Rate

6. RETURN ROUND(result, DecimalPlaces)
```

Registration:

csharp

```
services.AddScoped<ICurrencyConverter, CurrencyConverter>();
```

C10 — Price Resolver: Currency-Aware Resolution

Problem: Addendum 45's `IPurchasePriceResolver` returns the pricelist's `CurrencyId` and `UnitPrice` directly. If the pricelist currency differs from the PO currency, the wrong amount is placed on the PO line.

Odoo Reference: Odoo's `_onchange_product_id` calls `seller.currency_id._convert(seller.price, self.currency_id, self.company_id, self.order_id.date_order.date())` — converting the seller's price from the seller's currency to the PO's currency using the order date.

Solution: Enhance `PriceResolutionResult` and add optional currency conversion.

Enhanced DTOs:

csharp

```
public class PriceResolutionRequest
{
    // ... existing fields from Addendum 45 ...
    public int OrgId { get; set; }
    public int ProductId { get; set; }
    public int VendorId { get; set; }
    public decimal Quantity { get; set; }
    public DateTime OrderDate { get; set; }
    public DateTime? ExpectedDeliveryDate { get; set; }

    // NEW: currency context for conversion
    public int? POCurrencyId { get; set; }   // If provided, convert resolved price to th
}

public class PriceResolutionResult
{
    // ... existing fields from Addendum 45 ...
    public bool Found { get; set; }
    public decimal? UnitPrice { get; set; }              // In PO currency (converted if
    public int? CurrencyId { get; set; }                 // The PO currency (or pricelist
    public int? SupplierPricelistId { get; set; }
    public PricingDateBasis? DateBasisUsed { get; set; }
    public DateTime? EffectiveDateUsed { get; set; }
    public string WarningMessage { get; set; }

    // NEW: multi-currency tracking
    public decimal? OriginalPrice { get; set; }           // Price in pricelist's own cur
    public int? OriginalCurrencyId { get; set; }          // Pricelist's currency
    public decimal? ExchangeRateUsed { get; set; }        // Rate applied (null if same c
    public DateTime? ExchangeRateDate { get; set; }       // Date the rate was from
    public bool CurrencyConverted { get; set; }           // True if conversion was appli
    public bool CurrencyMismatchWarning { get; set; }     // True if conversion failed (n
}
```

Enhanced Resolution Algorithm:

```
... (steps 1–3 from Addendum 45 unchanged) ...

4. MATCH FOUND — check currency:

   IF request.POCurrencyId IS NOT NULL
      AND match.CurrencyId ≠ request.POCurrencyId:

      a. Call ICurrencyConverter.ConvertAsync(
            Amount = match.UnitPrice,
            FromCurrencyId = match.CurrencyId,
            ToCurrencyId = request.POCurrencyId,
            EffectiveDate = effectiveDateUsed,
            OrgId = request.OrgId
         )

      b. IF conversion succeeds:
            result.UnitPrice = convertedAmount
            result.CurrencyId = request.POCurrencyId
            result.OriginalPrice = match.UnitPrice
            result.OriginalCurrencyId = match.CurrencyId
            result.ExchangeRateUsed = rateUsed
            result.ExchangeRateDate = rateDate
            result.CurrencyConverted = true

      c. IF conversion fails (no exchange rate):
            result.Found = true  // we found a price, but can't convert
            result.CurrencyMismatchWarning = true
            result.UnitPrice = null  // don't place wrong-currency price
            result.OriginalPrice = match.UnitPrice
            result.OriginalCurrencyId = match.CurrencyId
            result.WarningMessage = "Price found: {OriginalPrice}
{OriginalCurrencyCode}
               but no exchange rate available for {PricelistCurrencyCode} →
               {POCurrencyCode} on {effectiveDate}. Enter price manually."

   ELSE (same currency or no PO currency):
      result.UnitPrice = match.UnitPrice
      result.CurrencyId = match.CurrencyId
      result.CurrencyConverted = false
```

PO Line — New Columns for Currency Tracking:

Column	Type	Null	Default	Description
`OriginalCurrencyId`	int	YES	NULL	Pricelist's currency (before conversion)
`OriginalCurrencyPrice`	decimal(18,4)	YES	NULL	Price in pricelist's own currency
`ExchangeRateUsed`	decimal(18,6)	YES	NULL	Rate applied at resolution time

Migration M8:

```sql
ALTER TABLE purchase.PurchaseOrderLines
    ADD OriginalCurrencyId      INT             NULL,
        OriginalCurrencyPrice   DECIMAL(18,4)   NULL,
        ExchangeRateUsed        DECIMAL(18,6)   NULL;

ALTER TABLE purchase.PurchaseOrderLines
    ADD CONSTRAINT FK_POLines_OriginalCurrency
        FOREIGN KEY (OriginalCurrencyId)
        REFERENCES core.Currencies(Id);
```

UI — PO Line Currency Info:

When a price was converted from a different currency, show an info tooltip next to the price:

```
  Unit Price: [$54.35]  🏷️ Pricelist  ℹ️
                                  │
                                  └→ Tooltip: "Original: €50.00 @ 1.0870 (2026-
10-17)"
```

C11 — Resolver Query Plan Verification

Problem: The covering index `IX_SupplierPricelists_Lookup` was designed for the resolver query but has never been verified against an actual execution plan. Without verification, the index may not be used (wrong column order, missing INCLUDEs, parameter sniffing).

Verification Script:

```sql
```

```sql
-- ============================================================
-- Resolver Query Plan Verification
-- Run AFTER Migration M1 (Addendum 45) against dev database
-- with at least 10,000 pricelist records seeded
-- ============================================================

-- 1. Seed test data (if needed)
-- Insert 1,000 products × 3 vendors × 4 qty tiers × 2 date ranges = 24,000 rows
-- (Use separate seed script — not included here)

-- 2. Capture query plan + IO statistics
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

DECLARE @OrgId int = 1,
        @ProductId int = 500,
        @VendorId int = 25,
        @Qty decimal(18,4) = 75.0000,
        @OrderDate date = '2026-10-17',
        @DeliveryDate date = '2026-11-15';

-- Core resolver query from Addendum 45, Section 4, C3
SELECT TOP 1
    sp.Id,
    sp.UnitPrice,
    sp.CurrencyId,
    sp.PricingDateBasis,
    sp.MinQuantity
FROM purchase.SupplierPricelists sp
WHERE sp.OrgId = @OrgId
  AND sp.ProductId = @ProductId
  AND sp.VendorId = @VendorId
  AND sp.IsActive = 1
  AND sp.MinQuantity <= @Qty
  AND (
      (sp.PricingDateBasis = 0 AND @OrderDate BETWEEN sp.ValidFrom AND sp.ValidTo)
      OR
      (sp.PricingDateBasis = 1 AND COALESCE(@DeliveryDate, @OrderDate)
                                   BETWEEN sp.ValidFrom AND sp.ValidTo)
  )
ORDER BY sp.Priority ASC, sp.MinQuantity DESC;
```

```
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Expected Results:

Metric	Target	Action if missed
Logical reads	⩽ 5 pages	Check index column order
Scan type	Index Seek	Verify leading columns match WHERE
Key lookups	0 (all columns in INCLUDE)	Add missing columns to INCLUDE
Execution time	< 1ms	Acceptable up to 5ms under load
OR branch handling	Two seeks concatenated or filter	Acceptable — OR on PricingDateBasis

Load Test (50 concurrent resolutions):

csharp

```csharp
// Run via integration test
[Fact]
public async Task PriceResolver_ConcurrentAccess_NoBlocking()
{
    var tasks = Enumerable.Range(1, 50).Select(i =>
        _resolver.ResolveAsync(new PriceResolutionRequest
        {
            OrgId = 1,
            ProductId = (i % 100) + 1,
            VendorId = (i % 10) + 1,
            Quantity = 50 + (i * 10),
            OrderDate = DateTime.Today
        })
    );

    var results = await Task.WhenAll(tasks);

    // All should complete without deadlock or timeout
    Assert.All(results, r => Assert.NotNull(r));
}
```

Index Tuning Notes:

The `OR` on `PricingDateBasis` in the resolver query may cause the optimizer to choose a scan instead of two seeks. If the plan shows a scan:

Option A — Split query: Run two separate queries (one for `PricingDateBasis = 0`, one for `PricingDateBasis = 1`) and take the best match. Each query becomes a simple seek.

Option B — Filtered indexes: Create two filtered indexes:

```sql
-- Filtered index for OrderDate-basis records
CREATE NONCLUSTERED INDEX IX_SupplierPricelists_OrderDateBasis
    ON purchase.SupplierPricelists (OrgId, ProductId, VendorId, ValidFrom, ValidTo)
    INCLUDE (UnitPrice, MinQuantity, [Priority], CurrencyId)
    WHERE IsActive = 1 AND PricingDateBasis = 0;

-- Filtered index for DeliveryDate-basis records
CREATE NONCLUSTERED INDEX IX_SupplierPricelists_DeliveryDateBasis
    ON purchase.SupplierPricelists (OrgId, ProductId, VendorId, ValidFrom, ValidTo)
    INCLUDE (UnitPrice, MinQuantity, [Priority], CurrencyId)
    WHERE IsActive = 1 AND PricingDateBasis = 1;
```

Decision: Only add filtered indexes if the query plan verification shows a scan. The single covering index should handle the volume (≤ 100K rows typical for pricelist data).

---

5. Database Migrations

ID	Description	Schema	Type	Rollback
M4	Widen `PurchaseOrderLines.UnitPrice` from `decimal(18,2)` to `decimal(18,4)` (C3)	`purchase`	DDL	ALTER COLUMN back to (18,2)
M5	Widen `CK_POLines_PriceSource` to include `BlanketOrder (4)` (C5)	`purchase`	DDL	Restore original CHECK
M6	Add `DefaultPurchaseCurrencyId` to `core.BusinessPartners` (C6)	`core`	DDL	DROP COLUMN
M7	Create `finance.ExchangeRates` table with FKs, constraints, and indexes (C8)	`finance`	DDL	DROP TABLE
M8	Add `OriginalCurrencyId`, `OriginalCurrencyPrice`, `ExchangeRateUsed` to PurchaseOrderLines (C10)	`purchase`	DDL	DROP COLUMN ×3
M9	Add `OrderDate` to `purchase.PurchaseOrders` (C1)	`purchase`	DDL	DROP COLUMN
M10	Backfill `PurchaseOrders.OrderDate` from `CreatedAt` for existing POs	`purchase`	DML	No-op

Migration M9 — DDL:

```sql
ALTER TABLE purchase.PurchaseOrders
    ADD OrderDate DATE NOT NULL
        CONSTRAINT DF_PurchaseOrders_OrderDate DEFAULT CAST(SYSUTCDATETIME() AS date);
```

Migration M10 — DML:

```sql
-- Backfill existing POs: set OrderDate = date part of CreatedAt
UPDATE purchase.PurchaseOrders
SET OrderDate = CAST(CreatedAt AS date)
WHERE OrderDate = CAST(SYSUTCDATETIME() AS date);  -- only those that got today's default
```

Migration Execution Order:

6. API Endpoints

6.1 Exchange Rate CRUD

Method	Endpoint	Description
GET	`/api/finance/exchange-rates`	List with filters: `?currencyId=&from=&to=&source=`
GET	`/api/finance/exchange-rates/{id}`	Get single rate record
POST	`/api/finance/exchange-rates`	Create manual rate entry
PUT	`/api/finance/exchange-rates/{id}`	Update rate (manual entries only; auto-fetched are read-only)
DELETE	`/api/finance/exchange-rates/{id}`	Delete rate (manual entries only)

6.2 Currency Conversion

Method	Endpoint	Description
POST	`/api/finance/convert-currency`	Convert amount between currencies at a given date

Request:

```
json
```

```
{
  "amount": 50.0000,
  "fromCurrencyId": 2,
  "toCurrencyId": 1,
  "effectiveDate": "2026-10-17"
}
```

Response:

```
json

{
  "success": true,
  "convertedAmount": 54.3500,
  "rateUsed": 1.087000,
  "rateDate": "2026-10-17",
  "fromCurrencyCode": "EUR",
  "toCurrencyCode": "USD",
  "errorMessage": null
}
```

6.3 Enhanced Price Resolution (replaces Addendum 45 section 6.2)

The existing `POST /api/purchase/resolve-price` endpoint is extended to accept optional `poCurrencyId`:

Updated Request:

```
json

{
  "productId": 42,
  "vendorId": 15,
  "quantity": 200,
  "orderDate": "2026-10-17",
  "expectedDeliveryDate": "2026-11-15",
  "poCurrencyId": 1
}
```

Updated Response (with currency conversion):

```
json
```

```
{
  "found": true,
  "unitPrice": 54.3500,
  "currencyId": 1,
  "currencyCode": "USD",
  "supplierPricelistId": 87,
  "dateBasisUsed": "OrderDate",
  "effectiveDateUsed": "2026-10-17",
  "warningMessage": null,
  "originalPrice": 50.0000,
  "originalCurrencyId": 2,
  "originalCurrencyCode": "EUR",
  "exchangeRateUsed": 1.087000,
  "exchangeRateDate": "2026-10-17",
  "currencyConverted": true,
  "currencyMismatchWarning": false
}
```

7. Business Rules

ID	Rule	Enforcement	Gap
BR-20	`PurchaseOrders.OrderDate` defaults to today (UTC) on PO creation and is editable only while PO is in Draft status	Frontend + Backend	1
BR-21	`PriceResolutionRequest.OrderDate` is always sourced from `PO.OrderDate`; the system never uses `DateTime.UtcNow` as a fallback for pricing	Backend	1
BR-22	When `OrderDate` is changed on a draft PO, fire the Addendum 45 C7 order-date re-lookup banner for all `PricingDateBasis = OrderDate` lines	Frontend + Backend	1
BR-23	RFQ prefill parameters (`vendorId`, `productId`, `quantity`) are individually optional; invalid values are silently ignored (no error toast, field stays empty)	Frontend	2
BR-24	PO line `UnitPrice` is stored at `decimal(18,4)`. UI displays 2 decimal places by default; full precision shown on hover/detail	Frontend + DB	3
BR-25	Override detection compares `UnitPrice` to `SystemResolvedPrice` directly (both `decimal(18,4)`) — no rounding tolerance needed	Backend	3
BR-26	Server-side price resolution is **mandatory** on PO line create and update — not optional or client-only. The `IPurchaseOrderLinePricingMiddleware` runs before persistence	Backend: Service	4
BR-27	Auto-generated PO lines (reorder point, purchase request conversion) call `IPurchasePriceResolver`. If not found: `PriceSource = Manual`, `UnitPrice = 0`	Backend: Factory	4,5
BR-28	`PriceSource = RFQ (3)` lines are never re-priced on any mutation. `PriceSource = BlanketOrder (4)` lines are never re-priced on any mutation	Backend: Middleware	4
BR-29	Every code path that creates PO lines MUST use `IPurchaseOrderLineFactory`. Direct entity instantiation is prohibited by architectural convention	Backend: Design	5
BR-30	When creating multiple PO lines at once (batch import, PR conversion), display a summary: "{N} priced, {M} require manual pricing"	Frontend	5

ID	Rule	Enforcement	Gap
BR-31	Vendor `DefaultPurchaseCurrencyId` defaults the pricelist `CurrencyId` when adding a new pricelist record for that vendor. Buyer can always change it	Frontend + Backend	6
BR-32	Exchange rates are stored as "1 base currency = X foreign currency." Rate lookup returns the most recent rate on or before the requested date	Backend: Service	6
BR-33	When pricelist currency ≠ PO currency and no exchange rate exists: resolver returns `Found = true` + `CurrencyMismatchWarning = true` + `UnitPrice = null`	Backend: Resolver	6
BR-34	A manual exchange rate (`Source = 0`) takes precedence over an auto-fetched rate (`Source = 1`) for the same OrgId + CurrencyId + EffectiveDate	Backend	6
BR-35	PO line stores `OriginalCurrencyPrice` and `ExchangeRateUsed` for full audit trail when currency conversion was applied	Backend	6
BR-36	Same-currency resolution (pricelist currency = PO currency) requires no exchange rate lookup and no conversion	Backend: Resolver	6
BR-37	Resolver query must produce an Index Seek on `IX_SupplierPricelists_Lookup` with 0 key lookups. If OR branch causes scan, split into two seek queries	Backend: Perf	7

8. Test Scenarios

8.1 Order Date (Gap 1)

ID	Scenario	Expected Result	Priority
TS-27	Create PO → OrderDate defaults to today	`PO.OrderDate = today (UTC)`	P0
TS-28	Edit OrderDate on draft PO → save	New date persisted, lines re-resolvable	P0
TS-29	Change OrderDate on confirmed PO	Field is read-only; change rejected	P0
TS-30	Change OrderDate → 3 Pricelist lines with OrderDate basis	Banner: "3 lines may have different prices" — [Re-lookup] / [Keep]	P1
TS-31	Resolver uses PO.OrderDate, not DateTime.UtcNow	Create PO on Oct 15, edit line on Oct 17 → price resolved for Oct 15	P0

8.2 RFQ Prefill (Gap 2)

ID	Scenario	Expected Result	Priority
TS-32	Click "Create RFQ" → RFQ page opens with vendor + product + qty	All three fields pre-populated	P1
TS-33	Navigate to RFQ create with invalid vendorId in query string	Vendor dropdown empty (no error toast)	P1
TS-34	Navigate to RFQ create with only vendorId (no product/qty)	Vendor pre-selected, product and qty empty	P1

8.3 Price Precision (Gap 3)

ID	Scenario	Expected Result	Priority
TS-35	Pricelist price $12.3456 → resolved to PO line	PO line UnitPrice = $12.3456 (no rounding)	P0
TS-36	Pricelist $12.3456 → buyer doesn't change → save PO	PriceSource = Pricelist (not Override). No false-trigger	P0
TS-37	UI displays $12.35 (2 decimals) but stores $12.3456	Hover tooltip shows full precision	P1

8.4 Server-Side Resolution (Gap 4)

ID	Scenario	Expected Result	Priority
TS-38	API: Create PO line with ProductId + VendorId, no price supplied	Server calls resolver. If found: PriceSource=Pricelist, price set	P0
TS-39	API: Update PO line quantity (PriceSource=Pricelist)	Server re-resolves. New tier price applied silently	P0
TS-40	API: Update PO line quantity (PriceSource=Override)	SystemResolvedPrice updated, buyer's UnitPrice preserved	P0
TS-41	API: Update PO line quantity (PriceSource=RFQ)	No re-resolution. Price unchanged	P0
TS-42	Auto-PO from reorder point → pricelist exists	Line created with PriceSource=Pricelist, correct price	P0
TS-43	Auto-PO from reorder point → no pricelist	Line created with PriceSource=Manual, UnitPrice=0	P0

8.5 Import / Factory (Gap 5)

ID	Scenario	Expected Result	Priority
TS-44	Convert 5-line Purchase Request to PO → 3 have pricelists	3 lines Pricelist, 2 lines Manual. Summary: "3 priced, 2 need manual"	P0
TS-45	Accept RFQ → create PO line	PriceSource=RFQ, quoted price used, no resolver call	P0
TS-46	CSV import: 4 lines, 2 with prices, 2 without	2 Manual with imported price, 2 attempt resolve → Pricelist or Manual	P1
TS-47	PO line split from partial GR (PriceSource=Pricelist, qty reduced)	New line inherits Pricelist source; if qty crosses tier → re-resolved	P1

8.6 Multi-Currency (Gap 6)

ID	Scenario	Expected Result	Priority
TS-48	Set vendor DefaultPurchaseCurrencyId = EUR → add pricelist	CurrencyId defaults to EUR (not org base USD)	P0
TS-49	Vendor has no DefaultPurchaseCurrencyId → add pricelist	CurrencyId defaults to org base currency (USD)	P0
TS-50	Pricelist in EUR, PO in USD, exchange rate exists	Price resolved + converted: original EUR price in tooltip	P0
TS-51	Pricelist in EUR, PO in USD, NO exchange rate for the date	Warning: "No exchange rate available for EUR → USD on {date}"	P0
TS-52	Pricelist in USD, PO in USD (same currency)	No conversion, no rate lookup needed	P0
TS-53	Create manual exchange rate → override auto-fetched rate for same date	Manual rate used for resolution, auto-fetched rate ignored	P1
TS-54	Rate lookup date: no rate on exact date → uses most recent earlier date	Rate from 2026-10-15 used when resolving on 2026-10-17	P0
TS-55	Cross-rate: Pricelist in EUR, PO in GBP, org base = USD	EUR → USD → GBP via two rates. Both rates tracked on PO line	P1
TS-56	PO line stores OriginalCurrencyPrice, ExchangeRateUsed correctly	Audit trail: €50.00 @ 1.087 → $54.35	P0

8.7 Resolver Performance (Gap 7)

ID	Scenario	Expected Result	Priority
TS-57	Resolver query on 24K-row table → capture execution plan	Index Seek on IX_SupplierPricelists_Lookup, 0 key lookups	P1
TS-58	50 concurrent resolver calls → no blocking	All 50 complete within 5 seconds total, no deadlocks	P1
TS-59	Single resolution on warm cache	< 1ms elapsed time	P2

9. Feature Coverage Matrix

Feature	Entity	Migration	API	Service	UI	Rules	Tests
PO OrderDate Field	C1	M9, M10	—	—	C1	BR-20, 21, 22	TS-27–31
RFQ Prefill Fix	—	—	—	—	C2	BR-23	TS-32–34
Price Precision Alignment	C3	M4	—	—	C3	BR-24, 25	TS-35–37
Server-Side Pricing	C4	—	—	IPurchaseOrderLinePricingMiddleware	—	BR-26, 28	TS-38–43
PO Line Factory	C5	M5	—	IPurchaseOrderLineFactory	—	BR-27, 29, 30	TS-44–47
Vendor Default Currency	C6	M6	—	—	C6	BR-31	TS-48, 49
Pricelist Currency Default	C7	—	—	—	C7	BR-31	TS-48, 49
Exchange Rate Table	C8	M7	6.1	—	—	BR-32, 34	TS-53, 54
Currency Conversion Service	C9	—	6.2	ICurrencyConverter	—	BR-32, 36	TS-52, 55
Currency-Aware Resolver	C10	M8	6.3	IPurchasePriceResolver (enhanced)	C10	BR-33, 35, 36	TS-50–56

Feature	Entity	Migration	API	Service	UI	Rules	Tests
Query Plan Verification	C11	—	—	—	—	BR-37	TS-57–59

10. Open Items / Future Considerations

#	Item	Description	Target
1	Auto-Fetch Exchange Rates	Hangfire job to pull daily rates from ECB / Open Exchange Rates and insert into `finance.ExchangeRates`	Phase 2 of this addendum
2	GL Posting in Dual Currency	`FinancialTransaction` DTO (Addendum 44) needs `TransactionCurrencyId`, `TransactionAmount`, `BaseCurrencyAmount`	Finance Addendum
3	Exchange Rate Gain/Loss	Automatic journal entries when vendor bill rate differs from PO rate	AR/AP Addendum
4	Period-End Revaluation	Wizard to revalue open foreign-currency payables/receivables at month-end rates	Financial Statements Add.
5	PO Header Currency Control	Full `CurrencyId` on PO header with vendor-defaulting and all-lines-in-same-currency enforcement	Purchase Module Addendum
6	Multi-Currency Reporting	Configurable reporting currency (group consolidation) — third amount column on journal entries	Financial Statements Add.
7	Blanket Order Price Source	Full Blanket Order / Purchase Agreement entity with locked prices and call-off PO generation	Future Purchase Addendum
8	Display Precision Configuration	Organization-level setting for price display decimals (2 or 4)	Future enhancement

End of Addendum 46